Multimodal visual understand+SLAM navigation + visual line patroling(Voice Version)

1. Course Content

1. Learn to use the robot's visual understanding, line tracking, SLAM navigation, and other complex functional cases.
2. Study the key source code newly introduced in the tutorial.
3. **Note: This section requires you to have completed the map file configuration as described in the previous section on [7.Multimodal visual understand+SLAM navigation]**

2. Preparation

2.1 Content Description

> This section uses the Jetson Orin Nano as an example. For users of the gimbal servo USB camera version, this is used as an example. For Raspberry Pi and Jetson Nano boards, you need to open a terminal on the host computer and enter the command to enter the Docker container. Once inside the Docker container, enter the commands mentioned in this section in the terminal. For instructions on entering the Docker container on the host computer, refer to **[01. Robot Configuration and Operation Guide] -- [5.Enter Docker (For JETSON Nano and RPi 5)]**. For RDKX5 and Orin boards, simply open a terminal and enter the commands mentioned in this section. >

📝 This example uses `model:"qwen/qwen2.5-vl-72b-instruct:free","qwen-vl-latest"`

⚠ The responses from the big model for the same test command may not be exactly the same and may differ slightly from the screenshots in the tutorial. If you need to increase or decrease the diversity of the big model's responses, refer to the section on configuring the decision-making big model parameters in the **[04.AI Large Model Development] -- [5.Deploy the RAG knowledge base]**.

⚡ It is recommended that you first try the previous visual example. This example adds voice functionality to the singleton, and while the functionality is largely the same, I will not elaborate on the program implementation, code debugging, or results in detail! ! !

3. Run Case

3.1 Starting the Program

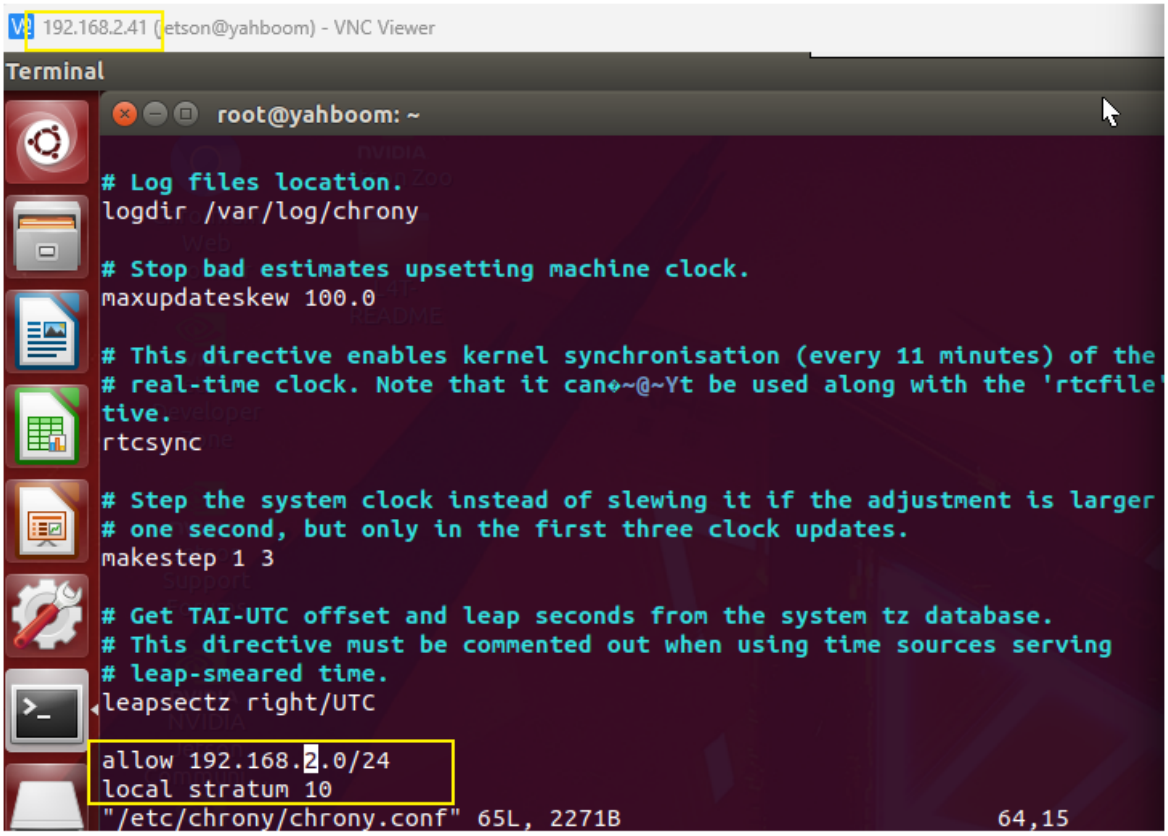
3.1.1 Jetson Nano board Startup Steps:

Due to Nano performance issues, navigation-related nodes must be run on a virtual machine. Therefore, navigation performance is highly dependent on the network. We recommend running in an indoor environment with a good network connection. You must first configure the following:

- **Board Side (Requires Docker)**

Open a terminal and enter

```
sudo vi /etc/chrony/chrony.conf
```

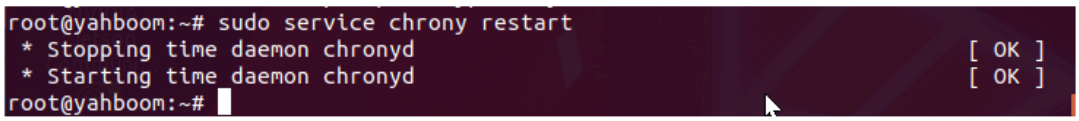


Add the following two lines to the end of the file: (Fill in 192.168.2.0/24 based on the actual network segment; this example uses the current board IP address of 192.168.2.41.)

```
allow 192.168.x.0/24 #x indicates the corresponding network segment
local stratum 10
```

After saving and exiting, enter the following command to take effect:

```
sudo service chrony restart
```



- **Virtual Machine**

Open a terminal and enter

```
echo "server 192.168.2.41 iburst" | sudo tee
/etc/chrony/sources.d/jetson.sources
```

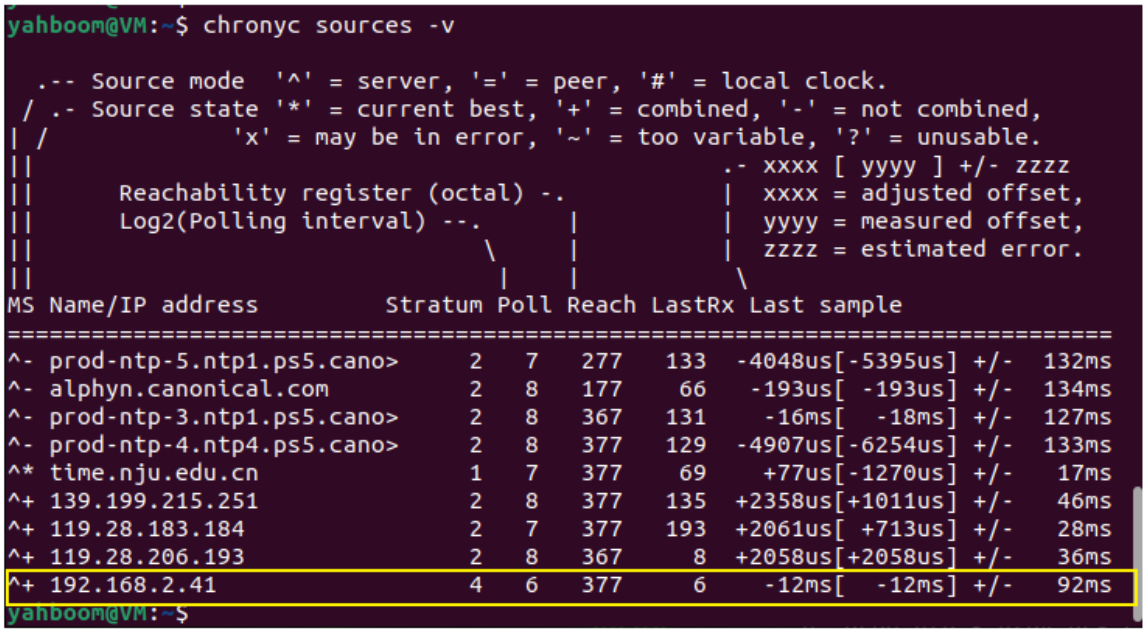
The 192.168.2.41 entry above is the board's IP address.

Enter the following command to take effect.

```
sudo chronyc reload sources
```

Enter the following command again to check the latency. If the IP address appears, it's normal.

```
chronyc sources -v
```



- **Start the program**

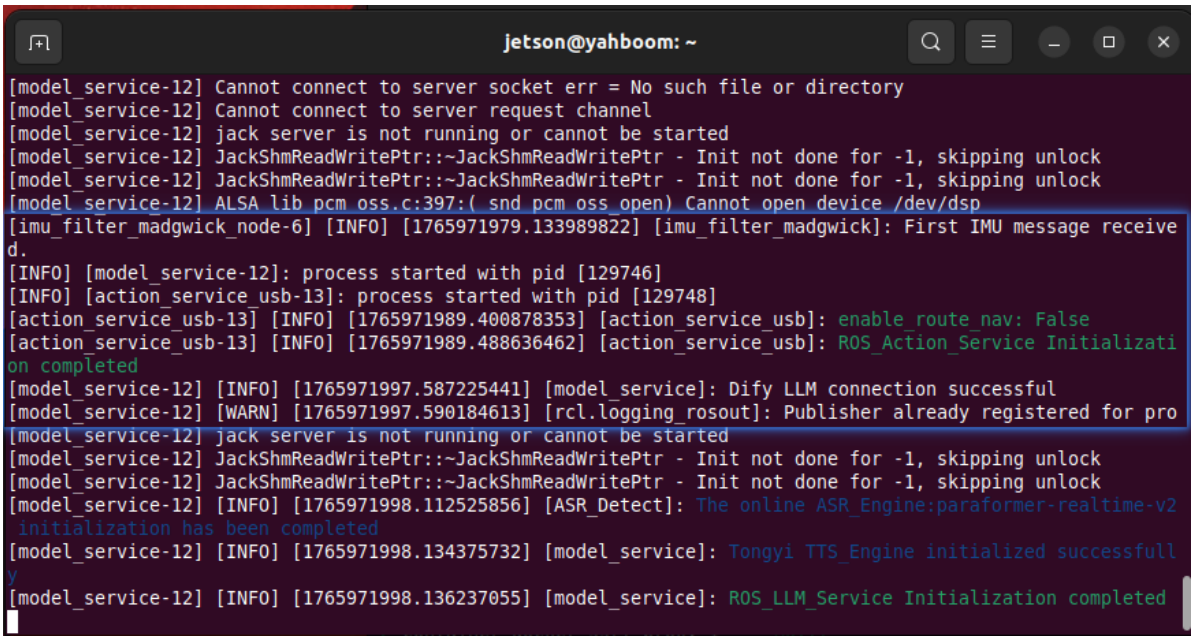
Open the host machine terminal and start the Dify environment:

```
sh ~/bringup_dify.sh
```

Open the terminal in Docker and enter the following command:

```
ros2 launch multi_brains llm_agent_control.launch.py
```

After initialization, the following content will be displayed:



On the virtual machine, create a new terminal and start.

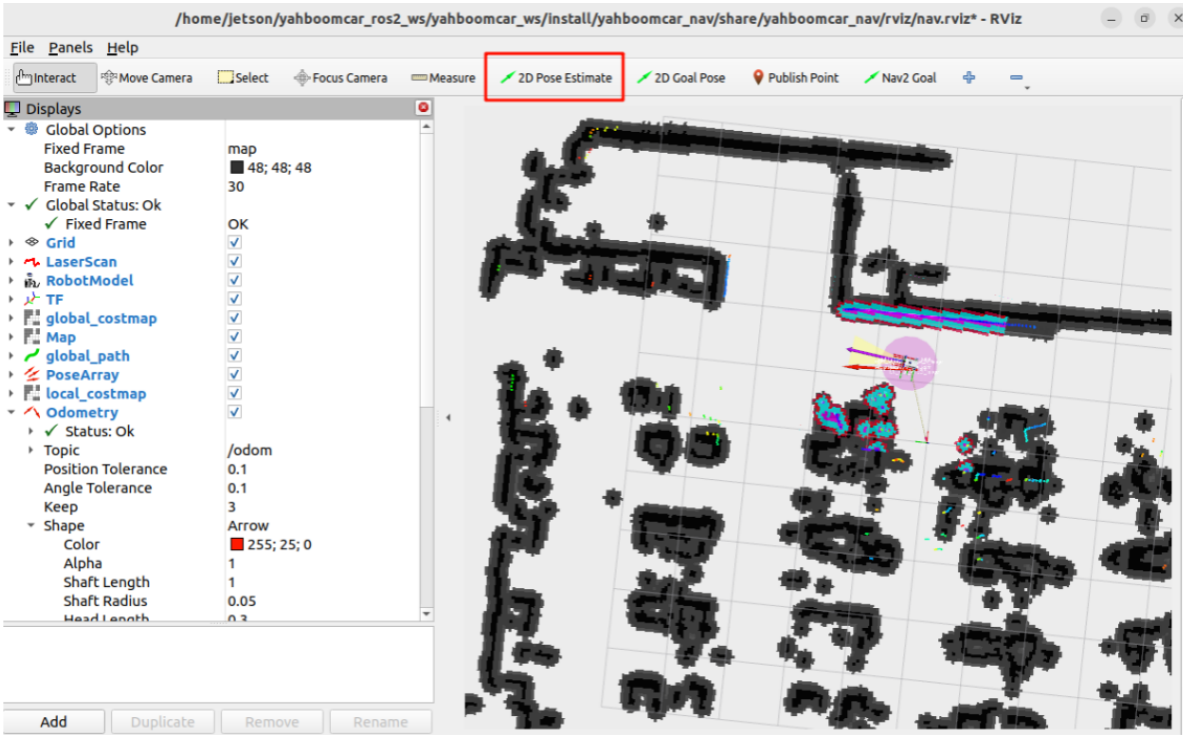
```
ros2 launch yahboomcar_nav display_nav_launch.py
```

Wait for the navigation algorithm to start before displaying the map. Then open a VM terminal and enter:

```
ros2 launch yahboomcar_nav navigation_teb_launch.py
```

Note: When running the car's mapping function, you must enter the save map command on the VM to save the navigation map.

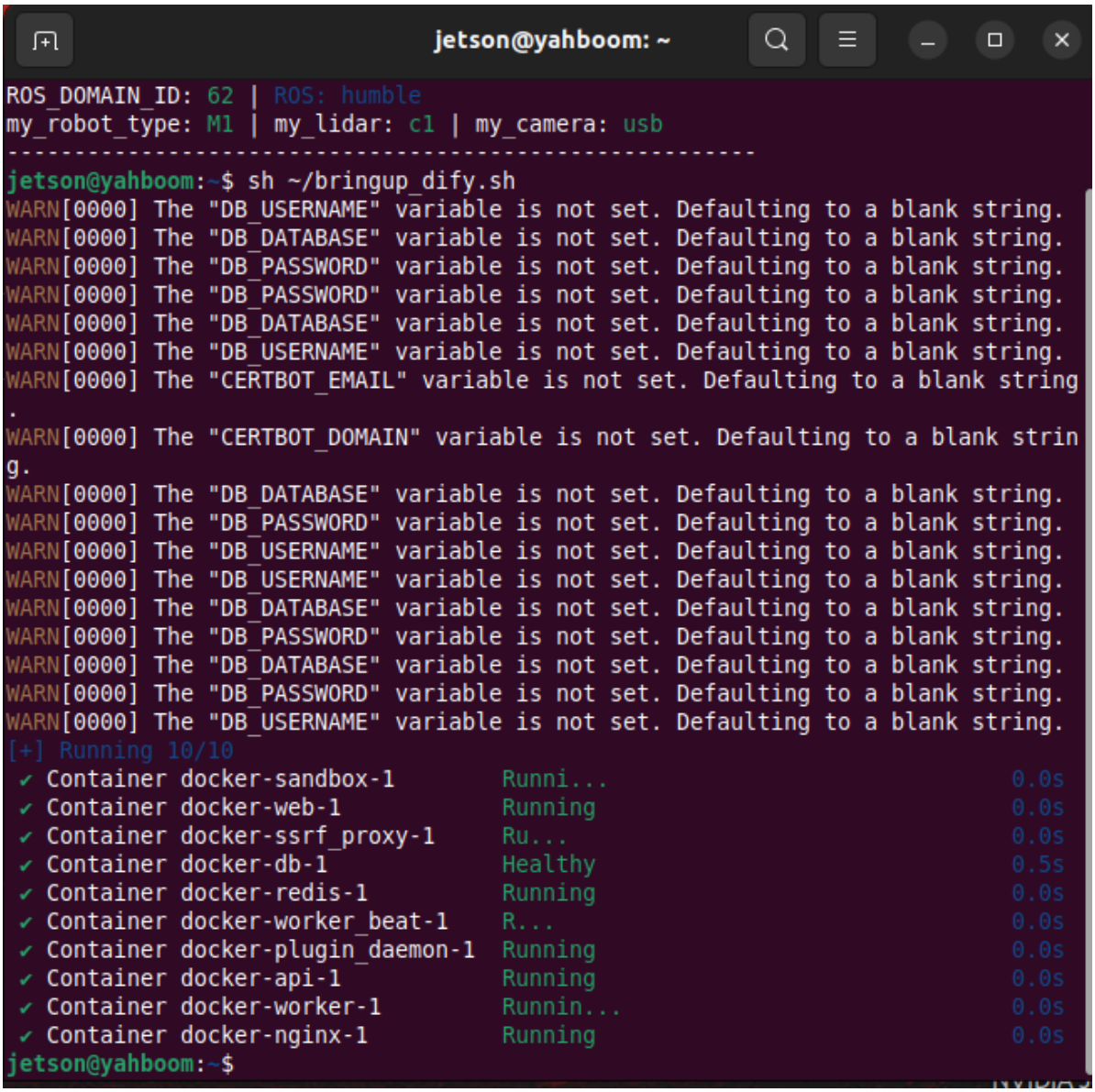
After that, follow the navigation function startup process to initialize positioning. The rviz2 visualization interface will open. Click **2D Pose Estimate** in the upper toolbar to select it. Roughly mark the robot's position and orientation on the map. After initializing positioning, preparations are complete.



3.1.1 RDKX5, Raspberry Pi PI5 and ORIN board startup steps:

Open the host terminal and start the Dify environment.

```
sh ~/bringup_dify.sh
```

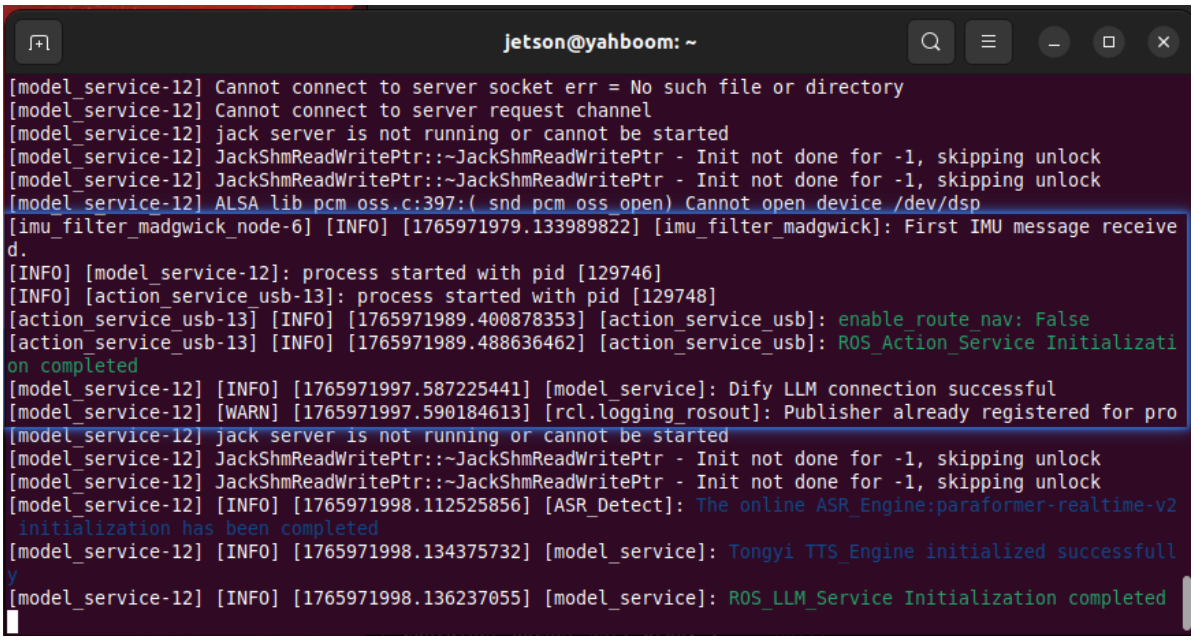


For Raspberry Pi 5, you need to first enter the Docker container. For RDKX5 and Orin board, this is not necessary.

Open a terminal in Docker and enter the command:

```
ros2 launch multi_brains llm_agent_control.launch.py
```

After initialization is complete, the following will be displayed:



Create a new terminal on the virtual machine and start the program.**(For Raspberry Pi and Jetson Nano, it is recommended to run the visualization in the virtual machine.**)

```
ros2 launch yahboomcar_nav display_nav_launch.py
```

Wait for the navigation algorithm to start before displaying the map. Then open the Docker terminal and type

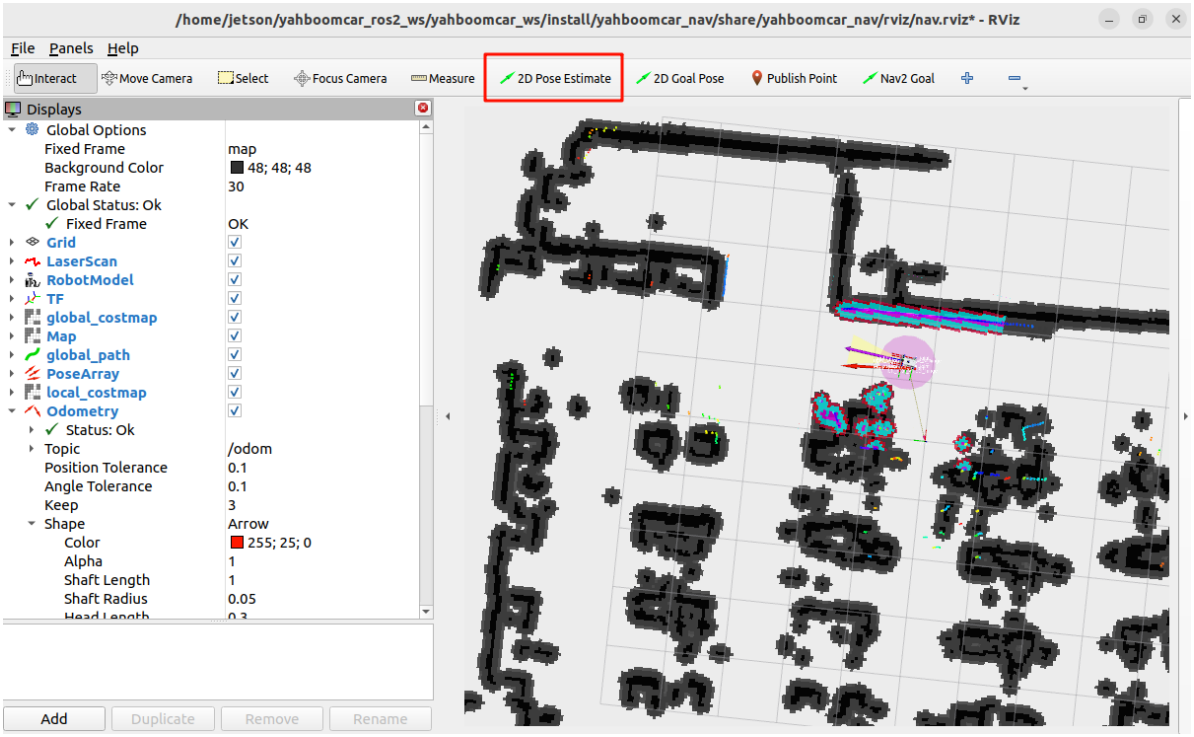
```
#Choose one of the two navigation algorithms
#Normal navigation
ros2 launch yahboomcar_nav navigation_teb_launch.py

#Fast Relocation Navigation (Not supported on RDKX5, Raspberry Pi 5, and Jetson Nano main controllers)
ros2 launch yahboomcar_nav localization_imu_odom.launch.py use_rviz:=false load_state_filename:=/home/jetson/yahboomcar_ros2_ws/yahboomcar_ws/src/yahboomcar_nav/maps/yahboomcar.pbstream

ros2 launch yahboomcar_nav navigation_cartodwb_launch.py maps:=/home/jetson/yahboomcar_ros2_ws/yahboomcar_ws/src/yahboomcar_nav/maps/yahboomcar.yaml
```

Note: yahboomcar.yaml and yahboomcar.pbstream must be mapped simultaneously, meaning they are the same map. Refer to the cartograph mapping algorithm to save the map.

After that, follow the navigation process to initialize positioning. The rviz2 visualization interface will open. Click **2D Pose Estimate** in the upper toolbar to select it. Roughly mark the robot's position and orientation on the map. After initial positioning, preparations are complete.



3.2 Test Case

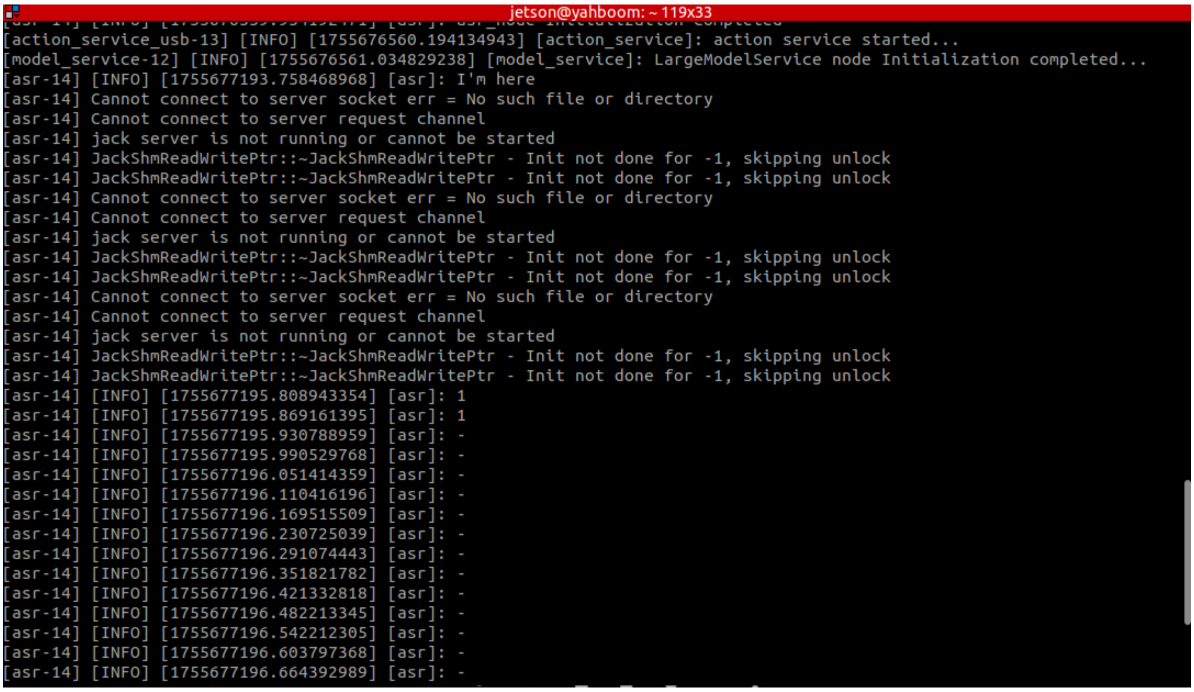
Here are some reference test cases; users can create their own dialogue commands.

- Move forward 0.5 meters and back 0.5 meters, record the current location, navigate to the tea room, and observe if there is a green line on the ground. If there is a green line, it will automatically patrol the green line. After completing the patrol, it will return to the starting point.

3.2.1 Example

First, use "Hi, yahboom" to wake the robot. The robot responds: "I'm here, please tell me." After the robot responds, the buzzer beeps briefly (beep-). The user can speak, and the robot will perform dynamic sound detection. If there is sound activity, it will print 1, and if there is no sound activity, it will print -. When the speech ends, it will perform tail tone detection. If there is silence for more than 450ms, the recording will stop.

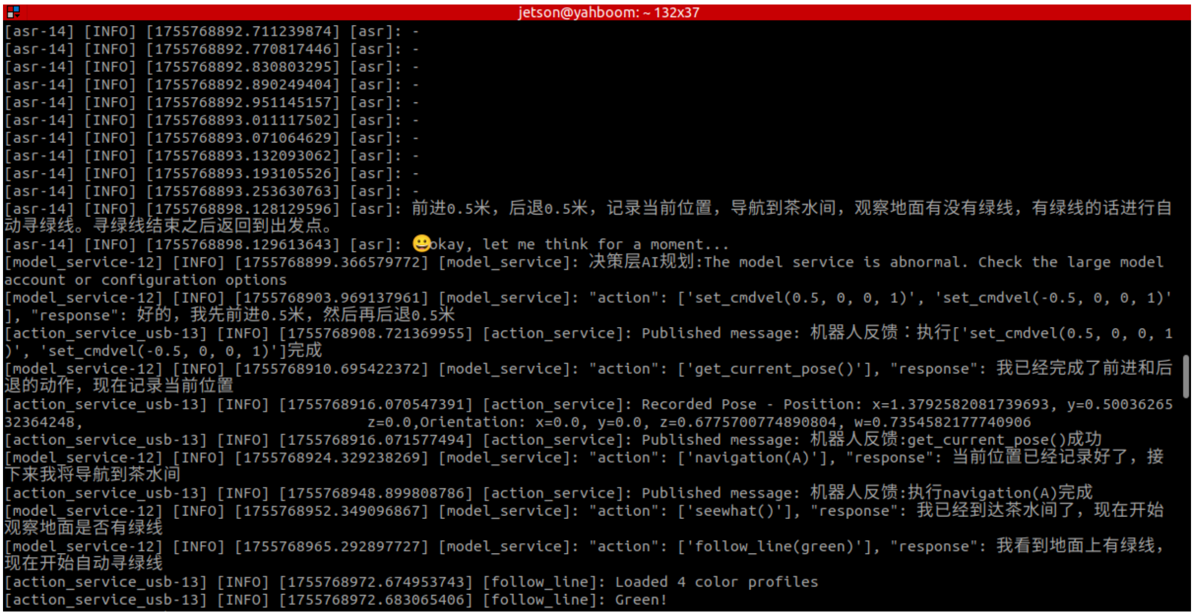
The following figure shows the dynamic voice detection (VAD):



The robot will first communicate with the user, then follow the instructions. The terminal will print the following message:

Network abnormality: Decision-making layer AI planning: The model service is abnormal. Check the large model account or configuration options. This does not affect the AI model's decision-making function, but it is recommended to try again under smooth network conditions!!

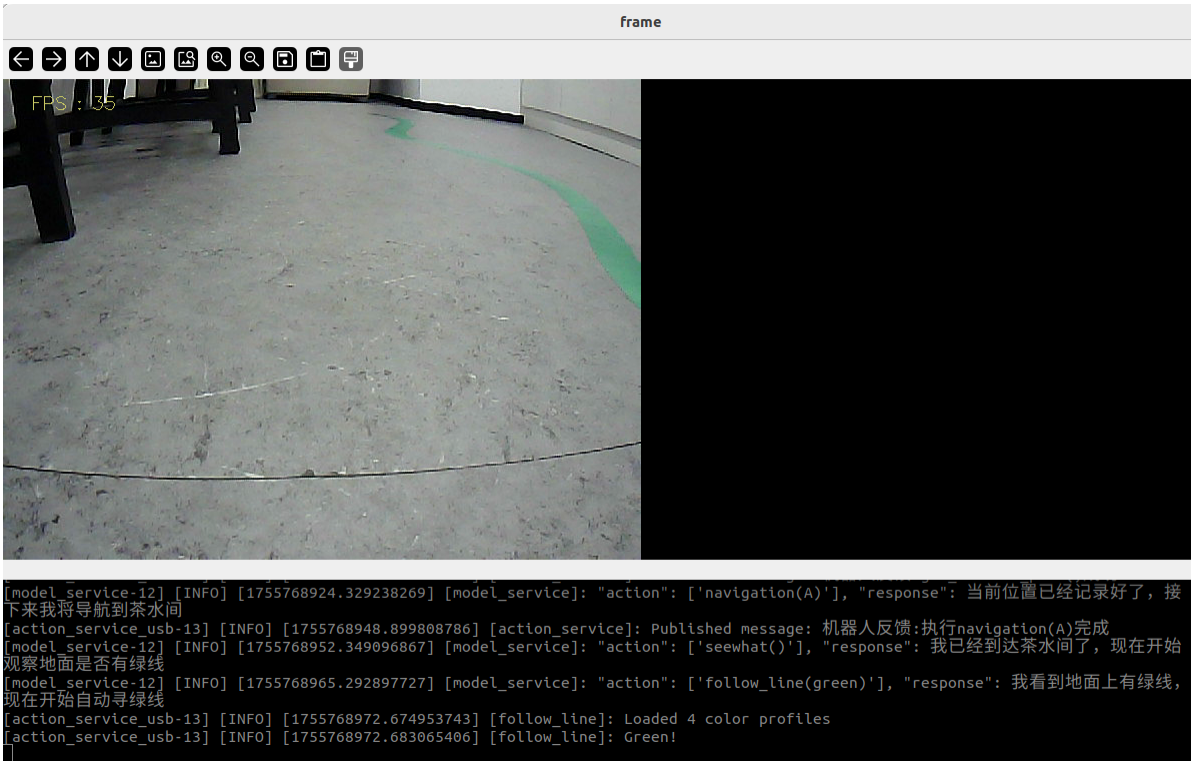
Normal: The decision-making layer AI planning will divide the steps 1, 2, 3, 4, etc.



`[model_service]: "action": ['seewhat()'], "response": I've arrived at the tea room. Now, I'm checking to see if there's a green line on the ground.`

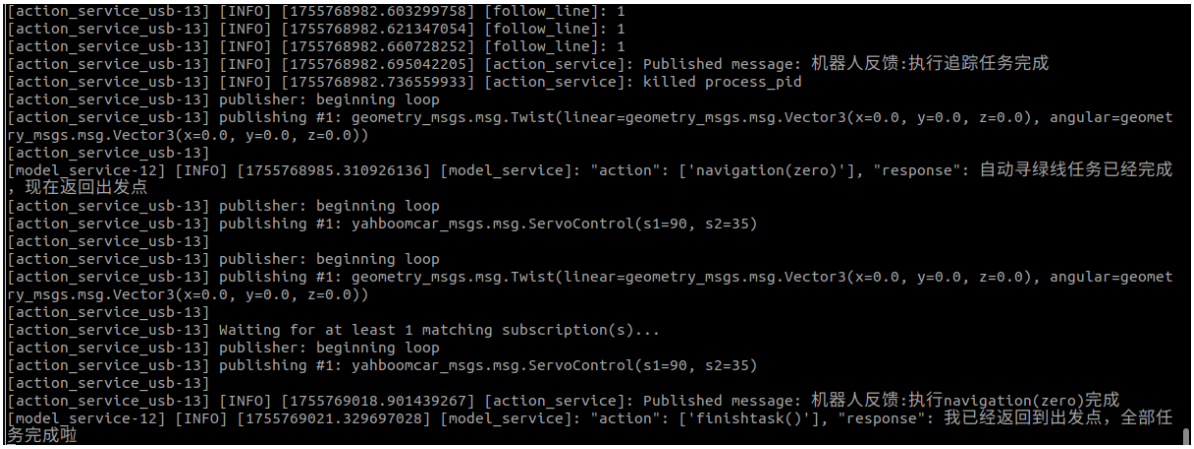
After reaching the navigation destination, this step calls the **seewhat** method. A window will open on the VNC screen and display for 5 seconds. An image is captured and uploaded to the large model for inference.

The large model receives the result, indicating that a green line has been observed, so the automatic green line detection program begins.

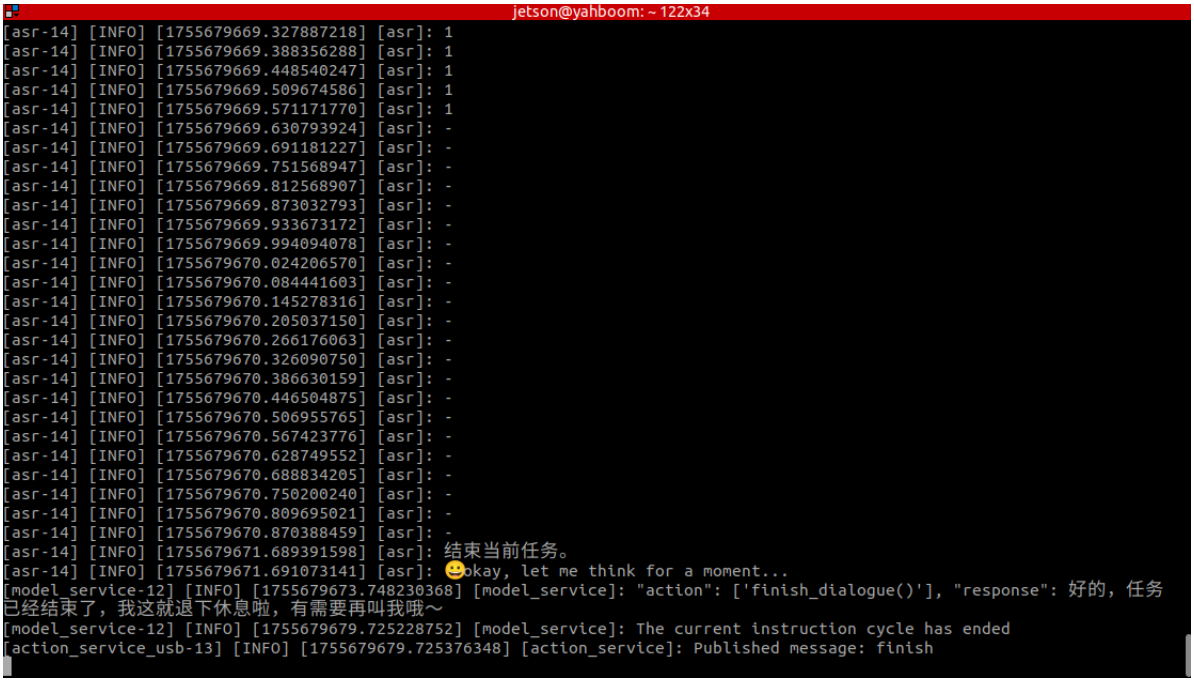


The logic is the same as in the previous example. If there is no target to track on the screen, the program will count down for 10 seconds, the terminal will print a 5-second countdown, and the process will automatically end, indicating that the task is complete.

Then the next command, `Return to Starting Point`, is executed.



After completing all tasks, the robot will enter the free conversation state again, but all conversation history will be retained. At this point, you can wakeyahboom up again and click "End Current Task" to end the current task cycle, clear the conversation history, and start a new one.



4. Source Code Analysis

Source code location:

Jetson Orin Nano:

```
#NUWA camera users
/home/jetson/yahboomcar_ros2_ws/yahboomcar_ws/src/multi_brains/multi_brains/action_service_nuwa.py
#USB camera users
/home/jetson/yahboomcar_ros2_ws/yahboomcar_ws/src/multi_brains/multi_brains/action_service_usb.py
```

Jetson Nano, Raspberry Pi:

You need to first enter Docker.

```
#NUWA Camera User
/root/yahboomcar_ros2_ws/yahboomcar_ws/src/multi_brains/multi_brains/action_serv
ice_nuwa.py
#USB Camera User
/root/yahboomcar_ros2_ws/yahboomcar_ws/src/multi_brains/multi_brains/action_serv
ice_usb.py
```

RDKX5：

```
#NUWA Camera User
/home/sunrise/yahboomcar_ros2_ws/yahboomcar_ws/src/multi_brains/multi_brains/act
ion_service_nuwa.py
#USB Camera User
/home/sunrise/yahboomcar_ros2_ws/yahboomcar_ws/src/multi_brains/multi_brains/act
ion_service_usb.py
```

This example uses the **seewhat**, **navigation**, **load_target_points**, **get_current_pose**, and **follow_line(self, color)** methods from the **CustomActionServer** class. Most of these methods have been explained in the **[7.Multimodal visual understand+SLAM navigation]** . For a detailed explanation of the **follow_line(self, color)** function, refer to the **[5.Multimodal visual understand + visual line patroling]**.